

CS 436 Cloud Computing — Project Report

Experts@SU — Faculty Experts Discovery Platform on AWS

Group Name

Experts@SU (55 BT 34)

Group Members

Kutluhan Berke Kılıçkaya

Hüseyin Eren Yıldız

Platform: Amazon Web Services (Region: eu-west-1, Ireland)

May 2026

Table of Contents

1. Introduction

2. High Level Design

2.1 Architecture Overview

2.2 Request Flow

2.3 System-Wide Decisions

3. Detailed Design — Cloud Services

3.1 VPC & Networking

3.2 EC2 (Compute)

3.3 RDS for PostgreSQL (Database / Storage)

3.4 Amazon S3 (Object Storage)

3.5 CloudFront (Content Delivery & HTTPS)

3.6 Application Load Balancer (Availability)

3.7 CloudWatch (Monitoring & Logging)

3.8 SNS (Notifications)

3.9 IAM (Access Control)

3.10 Security (Cross-Cutting)

3.11 Service Availability (Cross-Cutting)

3.12 Compatibility (Cross-Cutting)

4. Detailed Design — Non-Cloud Part

4.1 Technology Stack

4.2 Data Model & ETL

4.3 How the Solution Works

5. Solution Readiness & Demo

6. Contributions

1. Introduction

Experts@SU is a web platform that helps users discover the research expertise of Sabancı University faculty members. It ingests scholarly data (authors, publications, research topics and co-authorship relations) from the public OpenAlex catalogue, stores it in a relational database enriched with vector embeddings, and exposes it through an interactive web application. Users can browse faculty profiles, read publication lists, explore an interactive co-authorship network graph, view yearly citation/impact metrics, and run natural-language expert search (for example, "who works on lithium-battery thermal management?") that is answered with semantic vector similarity.

The aim of this project is to take a working multi-service application that previously ran only on a local developer machine (via Docker Compose against a local PostgreSQL instance) and deploy it as a robust, secure and observable system on Amazon Web Services. The end goal is a publicly reachable, always-on platform whose compute, storage, database, content delivery, monitoring and alerting are all provided by integrated AWS managed services, configured according to cloud best practices around security, availability and cost.

The dataset currently loaded into the production database contains 41,816 authors (256 of which are Sabancı University faculty), 24,673 publications, 20,392 research topics, 282,175 publication–topic links and 162,900 co-authorship edges. The platform is fully functional end-to-end on AWS and is reachable both over HTTP (directly) and HTTPS (through the content delivery network).

2. High Level Design

2.1 Architecture Overview

The solution is a classic three-tier web architecture (presentation, application, data) wrapped in AWS managed services for delivery, security, scaling-readiness and observability. All resources live inside a single dedicated Virtual Private Cloud (VPC) spanning two Availability Zones in the eu-west-1 (Ireland) region. The presentation and application tiers run as Docker containers on an EC2 instance placed in a public subnet; the data tier (PostgreSQL) runs as a managed RDS instance in isolated private subnets that have no route to the internet. An Application Load Balancer and a CloudFront distribution sit in front of the compute tier to provide health checking, a single entry point and HTTPS/CDN delivery. Static faculty photos are served from S3 through CloudFront. CloudWatch, SNS and IAM provide monitoring, alerting and least-privilege access.

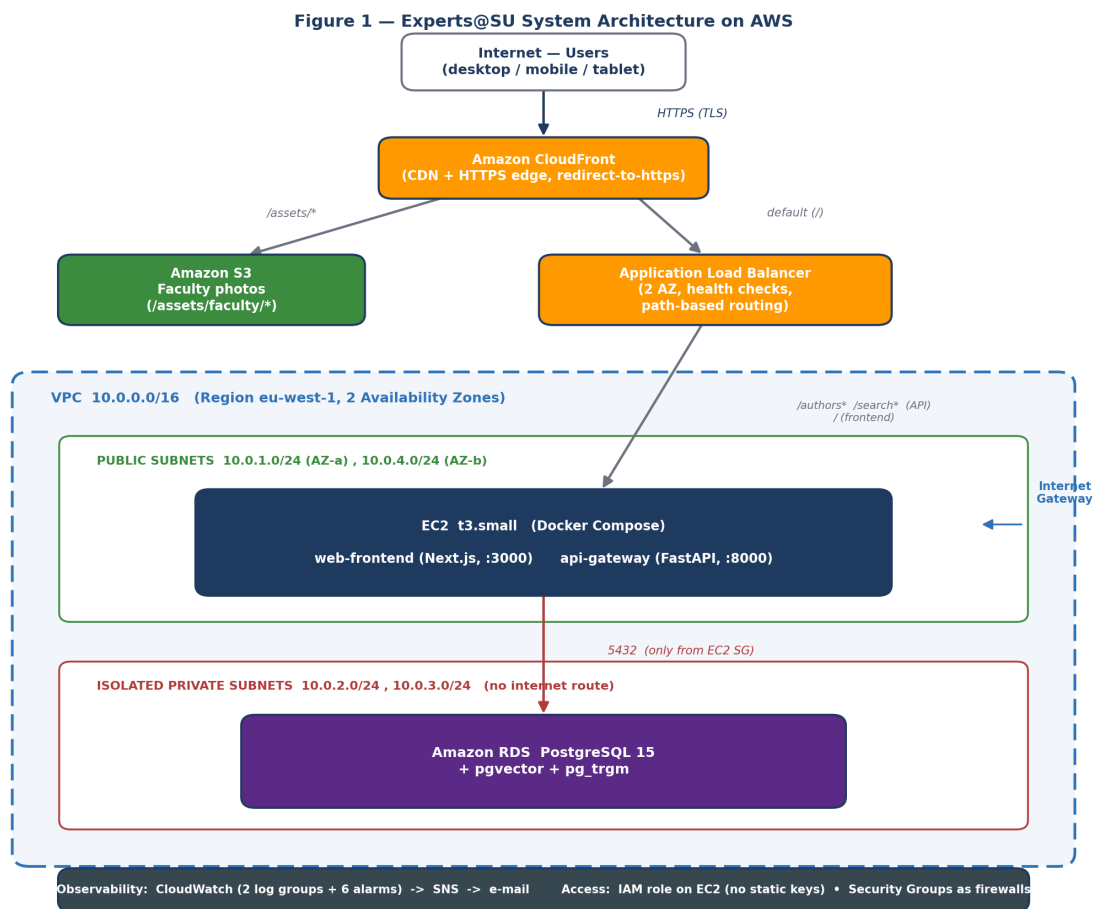


Figure 1 — Experts@SU System Architecture on AWS

2.2 Request Flow

- A browser requests the site. Over HTTPS the request hits CloudFront, which terminates TLS at the edge and forwards to the Application Load Balancer; over plain HTTP the ALB (or the EC2 instance directly) can be reached for testing.
- The ALB inspects the URL path: "/" and page routes go to the Next.js frontend container (port 3000); API paths (/authors*, /search*, /stats*, /docs) go to the FastAPI container (port 8000).
- The FastAPI service opens a connection pool to RDS PostgreSQL over the private network (port 5432, allowed only from the EC2 security group) and returns JSON.
- Faculty photos referenced in the JSON are loaded by the browser from CloudFront, which serves them from the S3 bucket and caches them at the edge.

2.3 System-Wide Decisions

Concern	Decision	Rationale (brief)
Availability	ALB health checks + container auto-restart + RDS backups + CDN caching + 2-AZ-ready subnets	Remove unhealthy targets, recover crashed containers, survive data loss, keep static content servable.
Security	Private isolated DB subnets, security-group firewalls, IAM roles, HTTPS via CloudFront, parameterized SQL	Defence in depth: the database is never internet-reachable and no static credentials live on the host.
Scalability	Stateless containers behind an ALB across two AZs	The application tier can be horizontally scaled later without re-architecting.
Observability	CloudWatch logs + custom metrics + 6 alarms + SNS e-mail	Operators are notified by e-mail before users notice a problem.
Cost	Burstable t3 classes, single-AZ RDS, no NAT Gateway	Keep the footprint near free-tier while leaving a clear production upgrade path.

3. Detailed Design — Cloud Services

Each AWS service used in the system is described below with three points: why the service is used, how it was configured, and what the design options were and why the chosen option was selected. Cross-cutting concerns explicitly requested for grading (Security, Service Availability and Compatibility) are covered in their own subsections at the end.

3.1 VPC & Networking

Why this service is used.

A dedicated VPC gives the whole system a private, isolated network in which we control IP addressing, routing and firewalling. It is the foundation that lets us place the database where the internet cannot reach it while keeping the web tier public.

How it was configured.

- VPC CIDR 10.0.0.0/16, with DNS hostnames and resolution enabled.
- Four subnets across two Availability Zones (eu-west-1a / eu-west-1b):

Public	10.0.1.0/24	(1a)	-> EC2, ALB	(route to Internet Gateway)
Public	10.0.4.0/24	(1b)	-> ALB (2nd AZ)	(route to Internet Gateway)
Private	10.0.2.0/24	(1a)	-> RDS	(no internet route)
Private	10.0.3.0/24	(1b)	-> RDS subnet group	(no internet route)

- One Internet Gateway is attached to the VPC; the public route table sends 0.0.0.0/0 to the IGW. The private subnets have no 0.0.0.0/0 route, so RDS is unreachable from the internet.
- Three security groups act as stateful firewalls (details in §3.10).

Design options & why chosen.

- **IGW vs NAT Gateway:** We deliberately did NOT provision a NAT Gateway. A NAT GW is billed per hour plus per-GB and would only be needed if private-subnet resources had to reach the internet. Our only outbound need (the ETL job calling the OpenAlex API) runs on the EC2 instance, which already sits in a public subnet with direct IGW egress. Skipping the NAT Gateway removes a constant cost without losing any capability.
- **Public vs private placement:** the database is the most sensitive asset, so it goes into isolated private subnets; the web tier must be reachable, so it goes public behind the ALB/CDN. This is the standard secure two-tier split.
- **Security Groups vs Network ACLs:** we rely on security groups (stateful, attached to resources) as the primary control because they are easier to reason about and sufficient for this topology; the default subnet-level Network ACL is left in place as a secondary layer.

Latency & bandwidth.

Low latency is a genuine requirement because the platform is interactive: the co-authorship network graph, the topic galaxy and the as-you-type expert search all feel sluggish if responses are slow. Two choices address this. First, the region eu-west-1 (Ireland) is the closest low-cost

European region to our Turkish user base, keeping the base round-trip modest. Second, CloudFront serves cacheable content (faculty photos and static frontend assets) from an edge location near the user, so repeat traffic never crosses the Atlantic. Bandwidth needs are moderate rather than heavy: the API returns compact paginated JSON, the large dataset stays inside the database, and the only sizeable payloads — the faculty images — are offloaded to S3/CloudFront and cached, so they do not consume application-server bandwidth.

DNS.

The system relies on AWS-provided DNS names: each managed service (CloudFront, the ALB and the RDS endpoint) is reached through its AWS-assigned hostname, and the EC2 instance is pinned to a static Elastic IP. We deliberately did not provision a Route 53 hosted zone because the project does not yet own a custom domain; the AWS-managed names are stable and sufficient. Registering a custom domain in Route 53 (and pointing it at CloudFront with an ACM certificate) is the natural next step once a domain is available, and would not change the underlying topology.

3.2 EC2 (Compute)

Why this service is used.

EC2 provides the virtual machine that hosts the two application containers (frontend and API). We need a persistent, always-warm process because the API loads a sentence-transformer machine-learning model into memory and keeps a database connection pool open.

How it was configured.

- Instance type t3.small (2 vCPU, 2 GiB RAM), Ubuntu 22.04 LTS, 20 GiB gp2 EBS root.
- Docker and Docker Compose installed via user-data on first boot; the two services are defined in `docker-compose.prod.yml` and run with restart policy 'unless-stopped'.
- A static Elastic IP (54.217.203.47) is attached so the public address survives stop/start cycles.
- An IAM instance profile grants the host permission to ship logs and metrics to CloudWatch — no static AWS keys are stored on the box.

Design options & why chosen.

- **Containers vs bare VM processes:** we run the apps as Docker containers (not directly on the OS) for reproducible builds, dependency isolation between the Python API and the Node frontend, and one-command deploys.
- **EC2 + containers vs AWS Lambda (serverless):** serverless was evaluated and rejected for the API. The semantic-search model (all-MiniLM-L6-v2) is about 80 MB and takes seconds to load; on Lambda this would be paid as a cold start on every scale-out, and Lambda's memory/packaging limits make bundling the model awkward. A long-running container keeps the model and the asyncpg connection pool warm, giving consistently low latency. Serverless does remain a good fit for the occasional ETL batch.

- **t3.small sizing:** we started on t3.micro (1 GiB) but the Next.js production build and the ML model exhausted memory; t3.small (2 GiB) builds and runs comfortably while staying a low-cost burstable class.

3.3 RDS for PostgreSQL (Database / Storage)

Why this service is used.

The platform is data-centric: relational entities (authors, publications, topics, co-authorship edges), full-text search and 384-dimension vector embeddings all need to be stored and queried efficiently. A managed relational database removes the operational burden of backups, patching and failover.

How it was configured.

- Engine PostgreSQL 15.10 on db.t3.micro, 20 GiB gp2 storage, single-AZ.
- Placed in a DB subnet group spanning the two private subnets; not publicly accessible. Reachable only on port 5432 from the EC2 security group.
- Two extensions enabled: pgvector (vector similarity with an HNSW index for semantic search) and pg_trgm (trigram fuzzy matching for name search). Twelve schema migrations were applied and roughly 600k rows imported from the local database.
- Automated daily backups enabled (1-day retention, the free-tier maximum), giving point-in-time recovery within the window.

Design options & why chosen.

- **Managed RDS vs self-hosted DB on EC2:** we chose managed RDS so that backups, minor-version patching, monitoring integration and the option to enable Multi-AZ are handled by AWS rather than by us. A self-managed PostgreSQL container would have been a few dollars cheaper but would put durability and patching entirely on the team.
- **PostgreSQL+pgvector vs a dedicated vector DB:** PostgreSQL with pgvector keeps relational data, full-text search and vector search in a single engine and a single query, avoiding the cost and synchronisation complexity of running a separate vector store.
- **Single-AZ vs Multi-AZ:** single-AZ was chosen for cost (free tier). Multi-AZ (synchronous standby with automatic failover) is the documented production upgrade; the subnet group already spans two AZs, so enabling it is a one-click change.

Read/write profile & disk.

The workload is strongly read-heavy. Heavy writes happen only during the periodic ETL load (a bulk import that runs rarely), whereas day-to-day traffic is almost entirely reads: listing faculty, fetching profiles and publications, and running vector-similarity searches. This profile drove three decisions: (1) general-purpose gp2 SSD storage is adequate because we do not need the sustained high IOPS of provisioned-IOPS (io1/io2) volumes — the read queries are served largely from PostgreSQL's in-memory cache and from purpose-built indexes; (2) an HNSW index on the

embedding column makes vector search an approximate-nearest-neighbour lookup rather than a full scan, and trigram (pg_trgm) indexes accelerate fuzzy name search; (3) because the read pattern is repetitive, a read replica or an ElastiCache layer is the obvious scale-out path if traffic grows — but neither is needed at the current, low-concurrency scale, so they were intentionally left out to avoid unused cost.

3.4 Amazon S3 (Object Storage)

Why this service is used.

Faculty profile photos are static binary assets. Serving them from object storage behind a CDN offloads this traffic from the application server, makes the images available over our own HTTPS domain, and removes the original dependency on the external university web server.

How it was configured.

- A bucket holds 231 faculty photos under the `assets/faculty/` prefix (keyed by author id), uploaded with `image/jpeg` content-type and a 7-day cache-control header.
- A bucket policy grants `public s3:GetObject` only on the `assets/*` prefix; the bucket is registered as a CloudFront origin and CloudFront routes the `/assets/*` path to it.
- Each faculty record's image URL in the database was rewritten to the CloudFront asset URL, so the application now serves photos from our own infrastructure.

Design options & why chosen.

- **S3 vs storing images on the EC2 disk:** object storage is effectively unlimited, highly durable, and integrates natively with CloudFront, whereas the EBS volume is small, tied to one instance, and would be lost if the instance were replaced.
- **Public-read prefix vs Origin Access Control:** a scoped public-read policy on just `assets/*` is simple and adequate for non-sensitive public photos; Origin Access Control (locking the bucket to CloudFront only) is noted as a tightening step for production.

3.5 CloudFront (Content Delivery & HTTPS)

Why this service is used.

CloudFront provides TLS/HTTPS termination at edge locations, caching of static content close to users for low latency, and a single branded entry point that fronts both the S3 image origin and the dynamic application origin.

How it was configured.

- Two origins: an S3 origin (faculty photos) and a custom origin pointing at the ALB.
- Default behaviour forwards to the ALB origin; the `/assets/*` behaviour forwards to S3.
- Viewer protocol policy set to `redirect-to-https`, so plain HTTP requests are automatically upgraded to encrypted HTTPS.

Design options & why chosen.

- **CloudFront vs exposing the ALB directly:** the CDN gives us free managed TLS, edge caching and basic DDoS absorption (AWS Shield Standard) that a bare load balancer would not.
- **Single distribution with path-based origins:** one distribution with two path behaviours keeps everything under one domain and one TLS certificate.

3.6 Application Load Balancer (Availability)

Why this service is used.

The ALB is the single, stable entry point to the compute tier. It performs health checks, routes by URL path to the correct container, and is the component that makes future horizontal scaling possible without changing the rest of the system.

How it was configured.

- Internet-facing ALB across the two public subnets (two AZs).
- Two target groups — frontend (HTTP :3000) and API (HTTP :8000) — each with health checks (30 s interval, healthy after 2 checks, unhealthy after 3).
- A listener rule routes `/authors*`, `/search*`, `/stats*`, `/docs` and `/openapi.json` to the API target group; everything else defaults to the frontend target group.

Design options & why chosen.

- **ALB vs pointing CloudFront straight at the instance:** the ALB adds health-based fault isolation (a crashed container is taken out of rotation automatically) and a scaling seam, and it spans two AZs.
- **Application (L7) vs Network (L4) load balancer:** we need HTTP path-based routing to split frontend and API traffic, which only the layer-7 application load balancer provides.

3.7 CloudWatch (Monitoring & Logging)

Why this service is used.

CloudWatch is how we ensure nothing is silently wrong at runtime: it centralises container logs, collects host metrics, and triggers alarms when thresholds are breached.

How it was configured.

- Container logs are shipped with the Docker awslogs driver into two separate log groups, `/experts-su/api-gateway` and `/experts-su/web-frontend`, so each service's logs are independently searchable.
- The CloudWatch agent on EC2 publishes custom CPU, memory and disk metrics.
- Six alarms are defined, all wired to the SNS topic:

Alarm	Metric source	Condition
experts-su-high-cpu	AWS/EC2	CPU > 80% (5 min)
experts-su-high-memory	custom (agent)	memory > 85%
experts-su-high-disk	custom (agent)	root disk > 85%
experts-su-instance-down	AWS/EC2	status check failed
experts-su-rds-high-cpu	AWS/RDS	DB CPU > 80%
experts-su-rds-low-storage	AWS/RDS	free storage < 2 GB

Design options & why chosen.

- **awslogs driver vs tailing log files:** the driver gives clean, per-container log streams natively (an earlier file-tailing approach produced a single jumbled stream and was replaced).
- **Alarming on built-in + custom metrics:** built-in metrics cover CPU and status; memory and disk are not exposed by EC2 by default, so the agent supplies them — otherwise a full disk or a memory leak would go unnoticed.

3.8 SNS (Notifications)

Why this service is used.

SNS turns a CloudWatch alarm into an actual human notification. Without it, alarms would change state silently in the console.

How it was configured.

- A topic, experts-su-alerts, with a confirmed e-mail subscription as the endpoint.
- All six CloudWatch alarms publish to this topic on both ALARM and OK transitions, so operators are told both when something breaks and when it recovers.

Design options & why chosen.

- **E-mail subscription vs SMS/push:** e-mail is free, reliable and needs no extra integration; the topic can fan out to SMS or a chat webhook later without changing the alarms.

3.9 IAM (Access Control)

Why this service is used.

IAM lets the EC2 host talk to CloudWatch without embedding long-lived AWS access keys on the machine, following the principle of least privilege.

How it was configured.

- An IAM role (experts-su-ec2-cloudwatch) carrying the CloudWatch agent/logs policies is attached to the instance via an instance profile; the host obtains temporary, automatically-rotated credentials.

Design options & why chosen.

- **Instance role vs static keys in a file:** a role removes the risk of leaked credentials and the burden of rotating them manually.

3.10 Security (Cross-Cutting)

Security is handled in depth across several layers. The table maps common attack types to the concrete measure implemented in this system.

Attack / threat	Mitigation in this system
SQL injection	All DB access uses parameterized asyncpg queries; no string-built SQL.
Direct database compromise	RDS is in isolated private subnets with no internet route; port 5432 is allowed only from the EC2 security group.
Man-in-the-middle / eavesdropping	HTTPS via CloudFront with redirect-to-https; TLS terminates at the edge.
Credential theft from the host	No static AWS keys on EC2; access is via an IAM instance role with temporary credentials.
Over-broad network exposure	Least-privilege security groups: ALB opens 80/443; RDS opens 5432 only to the EC2 security group.
Volumetric DDoS	CloudFront edge + AWS Shield Standard absorb and disperse traffic.

Recommended hardening (future).

- Enable RDS encryption-at-rest (KMS) and move the DB password into AWS Secrets Manager.
- Add AWS WAF on CloudFront and tighten the S3 bucket with Origin Access Control.

3.11 Service Availability (Cross-Cutting)

Availability is addressed at every layer:

- Fault isolation: the ALB health-checks targets and removes an unhealthy container from rotation automatically.
- Self-healing: containers run with restart policy 'unless-stopped', so a crashed process is restarted by Docker.
- Durability: RDS automated daily backups give point-in-time recovery; S3 stores images with very high durability.
- Graceful degradation: CloudFront keeps serving cached static content even if the origin is briefly degraded.
- Proactive alerting: the instance-down and resource alarms e-mail operators before users are affected.
- Scale-readiness: the application tier is stateless behind a 2-AZ ALB, so adding instances (and enabling RDS Multi-AZ) is the clear next step toward full HA.

Honest current limitation: the system runs on a single EC2 instance and single-AZ RDS, so the compute and database tiers are currently single points of failure. The architecture was intentionally built so that removing these limits (an Auto Scaling group across AZs and Multi-AZ RDS) requires configuration changes only, not a redesign.

3.12 Compatibility (Cross-Cutting)

- Multiple device types: the frontend is a responsive web application, so it renders on desktop, tablet and mobile browsers from the same codebase.
- Multiple browsers: built with standard, evergreen-browser web technologies (HTML/CSS/ES modules) plus server-side rendering for fast first paint.
- CDN-level compatibility: CloudFront serves over HTTP/2 with compression and broad geographic edge coverage.
- Open integration surface: the backend is a documented REST/JSON API (auto-generated OpenAPI at [/docs](#)), so any other client or device can consume the same data.

4. Detailed Design — Non-Cloud Part

This section describes what was built at the application level, on top of the cloud services, and how the solution works.

4.1 Technology Stack

Layer	Technology	Why
Frontend	Next.js 16 / React 19, TypeScript, D3.js, Recharts, react-force-graph-2d	Server-side rendering for fast loads plus rich interactive data visualisation (network graph, charts).
Backend API	FastAPI (Python), Uvicorn, asyncpg	Async, high-throughput REST API with first-class typing and auto OpenAPI docs.
Semantic search	sentence-transformers (all-MiniLM-L6-v2, 384-dim), NumPy	Encodes queries and faculty text into vectors for meaning-based search.
Database	PostgreSQL 15 + pgvector + pg_trgm	Relational, vector and fuzzy search in one engine.
Data ingestion	Python ETL service against the OpenAlex API	Fetches and normalises authors, publications, topics and co-authorship data.
Packaging	Docker + Docker Compose	Reproducible, isolated, one-command deployment.

4.2 Data Model & ETL

The relational schema centres on authors and publications with join tables for the many-to-many relations. The ETL pipeline matches Sabancı faculty names to OpenAlex author IDs, downloads their publications and topics, computes co-authorship edges, and stores per-faculty text chunks together with their 384-dimension embeddings in a pgvector column indexed with HNSW for fast approximate nearest-neighbour search.

Principal tables (production row counts):

Table	Rows	Purpose
authors	41,816	All authors (256 flagged as SU faculty).
publications	24,673	Papers with title, year, venue, citations.
author_publications	111,963	Author–paper links.
topics	20,392	Research topics / subfields.
publication_topics	282,175	Paper–topic links.
coauthor_edges	162,900	Weighted co-authorship graph edges.

4.3 How the Solution Works

- Browse: the home page lists faculty with their photo, department and publication count, paginated from the API.
- Profile: each faculty page shows recent publications (sortable, searchable, paginated), yearly impact metrics (charts), a research-topic galaxy, and an interactive co-authorship network rendered with a force-directed graph.
- Expert search: a free-text query is classified for intent, embedded into a vector, and matched against faculty embeddings via pgvector cosine similarity; results are returned with explanatory snippets of the publications that matched.

5. Solution Readiness & Demo

The system is fully functional end-to-end on AWS. The production database is populated with the complete dataset, all containers are healthy, the load balancer reports healthy targets, monitoring is live and faculty photos are served from S3 via CloudFront.

Verified working features:

- Faculty listing with photos, departments and publication counts.
- Faculty detail pages: publications, yearly metrics, topic galaxy, co-authorship graph.
- Natural-language semantic expert search backed by pgvector.
- All six CloudWatch alarms in OK state; logs flowing into per-service log groups.

Access endpoints:

Channel	URL	Notes
Frontend (direct)	http://54.217.203.47:3000	EC2 over HTTP — recommended for the live demo.
Load Balancer	http://experts-su-alb-...elb.amazonaws.com	Path-routed entry to frontend + API.
CloudFront (HTTPS)	https://dlaow0tl93z78.cloudfront.net	CDN + HTTPS; also serves faculty photos.
API docs	http://54.217.203.47:8000/docs	Interactive OpenAPI documentation.

Known limitation for the demo: when the site is opened over the HTTPS CloudFront URL, the browser blocks the frontend's calls to the HTTP API as mixed content, so the live demo is best shown over the direct HTTP frontend endpoint. Routing the API through the same HTTPS origin is the planned fix.

6. Contributions

This section summarises each member's contribution across the formalisation of the idea, methodology/architecture design, data work, coding, validation and report writing.

Kutluhan Berke Kılıçkaya

- Cloud architecture design and the full AWS deployment: VPC/subnets/security groups, EC2, RDS, S3, CloudFront, ALB, CloudWatch, SNS and IAM provisioning and configuration.
- Database migration: pgvector/pg_trgm setup, schema migrations and import of the full (~600k-row) dataset into RDS.
- Monitoring and alerting design (custom metrics, six alarms, SNS e-mail) and end-to-end validation of the deployed system.
- Report writing.

Hüseyin Eren Yıldız

- Application development: the Next.js frontend (faculty listing, profile pages, publication list, metrics charts and the co-authorship graph) and the FastAPI backend endpoints.
- Helping AWS deployment.
- Data pipeline: the OpenAlex ETL, faculty name matching, embedding generation and the relational/vector data model.
- Semantic search implementation (intent detection + pgvector similarity) and functional testing of the application features.
- Report writing.